

# MOVIE THEATRE WEBSITE SOFTWARE DESIGN SPECIFICATION

Version 1.0

Group 21  
Nate Crivelli, Matthew Chung

Prepared for CS 250 - Introduction to Software Systems  
Instructor: Gus Hanna, Ph.D.

## SOFTWARE DESIGN SPECIFICATION =====

### SYSTEM OVERVIEW -----

The Movie Theatre Website is a web-based ticket purchasing and theatre management system for one theatre location. Customers can browse movies that are currently showing or coming soon, read movie details, select a showtime, choose available seats, select ticket types, pay through an outside payment service, and receive an order confirmation. Customers may purchase tickets as guests or create an account to view previous orders and manage basic profile information.

Authorized theatre administrators use a separate protected area to manage movies, theatres, showtimes, ticket types, prices, and orders. The system also communicates with a relational database, a third-party payment service, and an email service.

The design reflects the following important requirements from the SRS:

1. A seat cannot be sold to more than one customer for the same showtime.
2. A customer can temporarily reserve selected seats during checkout.
3. Temporary reservations expire if checkout is not completed.
4. An order is confirmed only after payment is approved.
5. Complete payment-card numbers and security codes are never stored.
6. Showtimes in the same theatre cannot overlap, including cleaning time.
7. Administrators can manage movie, theatre, showtime, price, and order data.
8. Registered customers can retrieve their order history.
9. Guests can retrieve an order by providing an order number and matching email.
10. The system supports one theatre location, English, and United States dollars in the first release.

## SOFTWARE ARCHITECTURE OVERVIEW

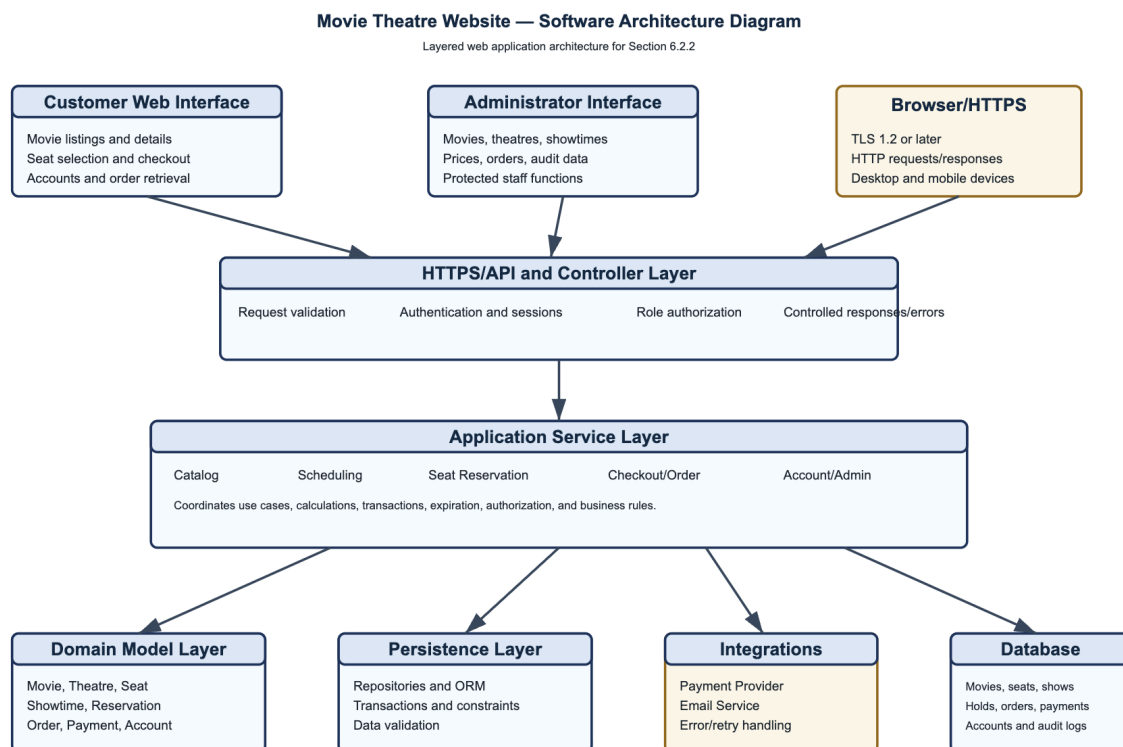
---

### Architectural style

The system uses a layered web application architecture. The presentation layer contains the public customer website and the protected administration website. The application layer coordinates use cases and enforces business rules. The domain layer represents movies, theatres, seats, showtimes, reservations, orders, payments, and user accounts. The persistence layer stores and retrieves data from a relational database. An integration layer communicates with the outside payment and email services.

The browser communicates with the application server through HTTPS using TLS 1.2 or later. The browser must not communicate directly with the database, payment provider, or email service.

### Software architecture diagram



### Component descriptions

## Customer Web Interface

---

Displays movie listings, movie details, showtimes, seating arrangements, checkout forms, confirmation pages, account pages, and order retrieval pages. It must clearly distinguish available, selected, reserved, sold, inaccessible, and accessible seats.

## Administrator Interface

---

Provides authorized theatre employees with pages to add, edit, publish, unpublish, and archive movies; add, edit, cancel, and view showtimes; manage ticket types and prices; view orders; and review important administrative changes.

## HTTPS/API and Controller Layer

---

Receives web requests, validates request data, establishes the current user or guest session, checks authorization, calls an application service, and returns an HTML or JSON response. This layer also converts service errors into clear, controlled messages.

## Application Services

---

Coordinate complete use cases. Services enforce rules such as showtime conflict detection, temporary seat holds, reservation expiration, payment approval, order confirmation, and administrator permissions.

## Domain Model

---

Contains the business objects and their validation behavior. The domain model does not depend on the browser or on a specific database implementation.

## Repositories/Persistence

---

Provide interfaces for storing and retrieving domain objects. Database transactions and uniqueness constraints must prevent two confirmed orders from selling the same seat for one showtime.

## External Integrations

---

The Payment Provider authorizes, captures, or reports payment results. The Email Service sends order-confirmation messages. The application stores only the outside payment reference and payment status, not complete card details.

-----

\_\_\_\_\_



```
- movieId: UUID
- title: String
- description: String
- rating: String
- runtimeMinutes: Integer
- releaseDate: LocalDate
- posterUrl: String
- trailerUrl: String
- status: MovieStatus
+ validate(): ValidationResult
+ publish(): void
+ unpublish(): void
+ archive(): void
+ updateDetails(details: MovieDetails): void
+ listShowtimes(date: LocalDate): List<Showtime>
}
```

```
CLASS Theatre {
- theatreId: UUID
- name: String
- seatingArrangement: String
- cleaningTimeMinutes: Integer
- active: Boolean
+ listSeats(): List<Seat>
+ isAvailable(start: DateTime, end: DateTime): Boolean
+ hasScheduleConflict(start: DateTime, end: DateTime): Boolean
+ updateDetails(name: String, arrangement: String, cleaningTime: Integer): void
}
```

```
CLASS Seat {
- seatId: UUID
- theatreId: UUID
- rowLabel: String
- seatNumber: Integer
- seatType: SeatType
- accessible: Boolean
- active: Boolean
+ getDisplayLabel(): String
+ canBeSold(): Boolean
+ getStatus(showtimeId: UUID): SeatStatus
}
```

```
CLASS Showtime {
- showtimeId: UUID
```

- movieId: UUID
- theatreId: UUID
- startTime: DateTime
- endTime: DateTime
- saleStartTime: DateTime
- saleEndTime: DateTime
- status: ShowtimeStatus
- priceScheduleId: UUID
- + validate(): ValidationResult
- + publish(): void
- + cancel(reason: String): void
- + hasConflict(other: Showtime, cleaningTimeMinutes: Integer): Boolean
- + getAvailableSeats(): List<Seat>
- + getSeatMap(): SeatMap

}

CLASS TicketType {

- ticketTypeId: UUID
- name: String
- description: String
- price: Decimal
- active: Boolean
- + validate(): ValidationResult
- + updatePrice(price: Decimal): void
- + deactivate(): void

}

CLASS PriceSchedule {

- priceScheduleId: UUID
- showtimeId: UUID
- ticketTypes: List<TicketType>
- taxRate: Decimal
- serviceFee: Decimal
- + getPrice(ticketTypeId: UUID): Decimal
- + calculateTax(subtotal: Decimal): Decimal
- + calculateFee(subtotal: Decimal): Decimal
- + calculateTotal(subtotal: Decimal, discount: Decimal): Decimal

}

CLASS SeatReservation {

- reservationId: UUID
- showtimeId: UUID
- seatIds: List<UUID>
- orderId: UUID?

```

- createdAt: DateTime
- expiresAt: DateTime
- status: ReservationStatus
+ create(showtimeId: UUID, seatIds: List<UUID>): SeatReservation
+ release(): void
+ expire(currentTime: DateTime): void
+ convertToSoldTickets(orderId: UUID): List<Ticket>
+ isActive(currentTime: DateTime): Boolean
}

```

```

CLASS Order {
- orderId: UUID
- confirmationNumber: String
- customerEmail: String
- userId: UUID?
- showtimeId: UUID
- tickets: List<Ticket>
- subtotal: Decimal
- fees: Decimal
- taxes: Decimal
- discount: Decimal
- total: Decimal
- status: OrderStatus
- createdAt: DateTime
- confirmedAt: DateTime?
+ calculateTotal(): Decimal
+ confirm(payment: Payment): void
+ cancel(reason: String): void
+ getDetails(): OrderDetails
+ matchesRetrievalData(number: String, email: String): Boolean
}

```

```

CLASS Ticket {
- ticketId: UUID
- orderId: UUID
- seatId: UUID
- ticketTypeId: UUID
- price: Decimal
- status: TicketStatus
+ validate(): Boolean
+ markSold(): void
+ refund(): void
}

```

```
CLASS Payment {  
  - paymentId: UUID  
  - orderId: UUID  
  - outsidePaymentReference: String  
  - amount: Decimal  
  - status: PaymentStatus  
  - authorizationDate: DateTime?  
  - failureReason: String?  
  + requestPayment(paymentData: PaymentData): PaymentResult  
  + recordApproval(reference: String, date: DateTime): void  
  + recordFailure(reason: String): void  
  + checkUnknownResult(): PaymentResult  
}
```

```
CLASS UserAccount {  
  - userId: UUID  
  - email: String  
  - passwordHash: String  
  - role: UserRole  
  - verificationStatus: VerificationStatus  
  - accountStatus: AccountStatus  
  - createdAt: DateTime  
  + authenticate(password: String): Boolean  
  + resetPassword(token: String, newPassword: String): Boolean  
  + updateProfile(profile: ProfileData): void  
  + isPermitted(action: AdminAction): Boolean  
}
```

```
CLASS CatalogService {  
  + listPublishedMovies(status: MovieStatus): List<Movie>  
  + searchByTitle(title: String): List<Movie>  
  + getMovieDetails(movieId: UUID): Movie  
  + listShowtimes(movieId: UUID, date: LocalDate): List<Showtime>  
}
```

```
CLASS SeatService {  
  + getSeatMap(showtimeId: UUID): SeatMap  
  + holdSeats(showtimeId: UUID, seatIds: List<UUID>): SeatReservation  
  + releaseReservation(reservationId: UUID): void  
  + expireReservations(currentTime: DateTime): Integer  
  + verifyAvailable(showtimeId: UUID, seatIds: List<UUID>): Boolean  
}
```

```
CLASS CheckoutService {
```

```

+ calculateOrder(reservationId: UUID, ticketSelections: List<TicketSelection>): OrderTotal
+ submitOrder(reservationId: UUID, customerData: CustomerData, paymentData:
PaymentData): OrderResult
+ handlePaymentFailure(orderId: UUID, result: PaymentResult): void
}

```

```

CLASS OrderService {
+ getOrder(orderId: UUID): Order
+ getCustomerOrders(userId: UUID): List<Order>
+ retrieveGuestOrder(number: String, email: String): Order
+ cancelOrder(orderId: UUID, requester: UserAccount): Boolean
}

```

```

CLASS PaymentService {
+ authorize(paymentData: PaymentData, amount: Decimal): PaymentResult
+ capture(reference: String, amount: Decimal): PaymentResult
+ refund(payment: Payment): PaymentResult
}

```

```

CLASS EmailService {
+ sendOrderConfirmation(order: Order): DeliveryResult
+ sendPaymentFailure(email: String, reason: String): DeliveryResult
+ sendCancellationNotice(order: Order): DeliveryResult
}

```

```

CLASS AdministrationService {
+ createMovie(movie: Movie, administrator: UserAccount): Movie
+ updateMovie(movie: Movie, administrator: UserAccount): Movie
+ archiveMovie(movieId: UUID, administrator: UserAccount): void
+ createShowtime(showtime: Showtime, administrator: UserAccount): Showtime
+ cancelShowtime(showtimeId: UUID, administrator: UserAccount): void
+ manageTicketType(type: TicketType, administrator: UserAccount): void
+ viewOrder(orderId: UUID, administrator: UserAccount): Order
}

```

```

CLASS AuditLog {
- auditId: UUID
- administratorId: UUID
- action: String
- entityType: String
- entityId: UUID
- changeSummary: String
- createdAt: DateTime
+ record(): void
}

```

```
}
```

```
INTERFACE PaymentProvider {  
    + authorize(amount: Decimal, paymentData: PaymentData): PaymentResult  
    + capture(reference: String, amount: Decimal): PaymentResult  
    + refund(reference: String, amount: Decimal): PaymentResult  
    + checkStatus(reference: String): PaymentResult  
}
```

```
INTERFACE Repository<T> {  
    + findById(id: UUID): T  
    + save(entity: T): T  
    + update(entity: T): T  
}
```

#### UML relationships and multiplicities

1. One Movie has zero or more Showtimes. Each Showtime belongs to exactly one Movie.
2. One Theatre has zero or more Seats. Each Seat belongs to exactly one Theatre.
3. One Theatre has zero or more Showtimes. Each Showtime belongs to exactly one Theatre.
4. One Showtime has one PriceSchedule and one or more possible Seats through its Theatre.
5. One Showtime has zero or more SeatReservations.
6. One SeatReservation contains one or more Seats and may be associated with one Order.
7. One Order contains one or more Tickets and may be associated with one registered UserAccount. Guest orders store an email without requiring a user account.
8. Each Ticket belongs to one Order, one Seat, and one TicketType.
9. One Order has zero or more Payment records, including possible retries or an unknown payment result.
10. CatalogService uses Movie and Showtime repositories.
11. SeatService uses Seat, Showtime, and SeatReservation repositories.
12. CheckoutService uses SeatService, PaymentService, OrderService, and EmailService.
13. AdministrationService uses Movie, Theatre, Showtime, TicketType, Order, and AuditLog repositories.
14. PaymentService uses the PaymentProvider interface.